```
-
-   describe "send_sms_message/1" do
-     test "successful request creates an SMS message" do
-       bypass = Bypass.open()
10      Process.put(:twilio_base_url, "http://localhost:#{bypass.port}")
-
-       resp = Jason.decode!(File.read!("test/support/fixtures/success.json"))
-
-       Bypass.expect_once(
15        bypass,
-         "POST",
-         "/Accounts/mock-account/Messages.json",
-         fn conn ->
-           conn
20          |> Plug.Conn.put_resp_header("Content-Type", "application/json")
-           |> Plug.Conn.resp(201, Jason.encode!(resp))
-         end
-       )
-
25      assert {:ok, message} = Conversations.send_sms_message(%NewMessage{})
-       assert message.from == resp["from"]
-       assert message.to == resp["to"]
-       assert message.body == resp["body"]
-       assert message.message_sid == resp["sid"]
30      assert message.account_sid == resp["account_sid"]
-       assert message.status == resp["status"]
-       assert message.direction == :outgoing
-     end
-
35    test "a failed request returns an error" do
-       bypass = Bypass.open()
-       Process.put(:twilio_base_url, "http://localhost:#{bypass.port}")
-
-       Bypass.expect_once(
40        bypass,
-         "POST",
-         "/Accounts/mock-account/Messages.json",
-         fn conn ->
-           Plug.Conn.resp(conn, 500, "")
45        end
-       )
-
-       assert Conversations.send_sms_message(%NewMessage{}) ==
-                {:error, "Failed to send message"}
50    end
-   end
- end
```

This test is fairly long, but the individual components are not complex. We use the Bypass library by calling `Bypass.open/0` on line 9. This creates a testing server endpoint, but it doesn't connect it to our request. Our `PhoneApp.Twilio.Api`